WHITE PAPER

Version 1.0 · October 2026

Invitation-Gated Federated Sign-In for Keycloak

Making an out-of-band invitation a precondition for corporate SSO, with no server-side extension, no database and no custom code inside the identity server

Ashwath

Project: `keycloak-invite-gate` · https://github.com/ashwath143/keycloak-invite-gate · Apache-2.0

Abstract

Federating Keycloak to a corporate directory answers *who* a person is, but not *whether they should be here*: by default every identity in the directory can reach every application behind the realm. This paper describes `keycloak-invite-gate`, a small Go service that turns an administrator's invitation into a precondition for federated sign-in. An invitation is stored as a *disabled* Keycloak user carrying the SHA-256 hash of a 256-bit, single-use, 15-minute nonce. Clicking the emailed link verifies the nonce, then enables the account and destroys the secret in one write. Sign-in runs through a dedicated first-broker-login flow in which Keycloak links an arriving identity to an existing account but never creates one. Reaching the application therefore needs two independent facts: an administrator invited this address, and a signature-verified assertion from the identity provider says the person holds it. The paper covers the mechanism and the Keycloak semantics it depends on, the architecture and the invitation state machine, a threat model with explicit trust assumptions, the verification strategy, the value against the alternatives, and an independent review with eight findings and a prioritised roadmap.

Language	Go 1.25, standard library only	**Keycloak**	26.5.x (pinned 26.5.4 in CI)
Size	2,560 lines of Go; 1,422 lines of tests	**Unit tests**	76 cases, clean under `-race`
Dependencies	0 third-party modules	**State**	none of its own; Keycloak is the record
Surface	5 HTTP endpoints, 3 mail providers	**Threat model**	10 threats, 6 documented gaps

Contents

Executive Summary 2

1 Introduction 3

2 Background 4

3 Problem Statement and Requirements 6

4 The Mechanism: Two Proofs, No Extension 7

5 System Architecture 10

6 The Invitation Lifecycle 12

7 Keycloak Integration 14

8 HTTP API and the Administrative Trust Boundary 17

9 Email Delivery 19

10 Security Analysis 20

11 Verification 23

12 Operations 26

13 Value Proposition 27

14 Roadmap and Conclusion 28

A Configuration Reference 29

B Glossary 30

References 30

Executive Summary

The problem. Organisations federate Keycloak to a corporate identity provider (Zoho Directory, Okta, Entra ID, any SAML 2.0 IdP) so that people sign in with the account they already have. The side effect is that the whole directory can now reach the application. For a 5,000-person company and an internal tool meant for 30 of them, that is the wrong default. The usual fixes are pre-creating every user by hand, writing and maintaining a Java authenticator plugin (an SPI), adopting a third-party extension, or adopting the Organizations model.

The approach. `keycloak-invite-gate` controls the realm from *outside* Keycloak, using only the Admin REST API:

1. An invitation creates a **disabled shadow user** whose attributes hold the hash of a one-time nonce, an expiry and a status. The invitation *is* that user record.
2. The invitee clicks an emailed link. The service verifies the nonce in constant time and then, in **one update**, enables the account and deletes the secret.
3. The identity provider is bound to a **dedicated first-broker-login flow**. Its two REQUIRED steps find an existing user and auto-link to it. Under Keycloak's documented rules, a REQUIRED step in a flow stops the ALTERNATIVE "create user" step from running. An uninvited identity finds no account and is refused; an invited but unaccepted one finds a disabled account and is refused.

What it brings.

- **Default-deny federated access** without running custom Java inside the identity server, so nothing has to be rebuilt against Keycloak internals at each upgrade.

- **No new system of record.** Invitation state lives in Keycloak user attributes, so there is no second store to drift out of sync with the identity provider.
- **A small, auditable trust surface**: about 2,500 lines of Go with no third-party modules. The security-critical rules sit in a 166-line package with no I/O.
- **Fail-closed defaults throughout.** SAML signature validation cannot be turned off, configuration errors stop the service from booting, an outage never opens the admin endpoints, and the flow requirements are corrected on every start.

What is proven, and what is not. The pure invitation core, the HTTP layer (against an in-memory directory), metadata parsing, configuration and mail are covered by 76 unit cases. An end-to-end suite, which the CI workflow runs against a real Keycloak 26.5.4, checks that provisioning is idempotent, that the invitation attributes persist, that acceptance enables the account and destroys the secret, and that the flow's requirement values read back as configured. The Keycloak behaviour the gate relies on was confirmed for this paper against the Keycloak 26.5.0 documentation and source. **No automated test yet drives a real SAML login through the broker**, so a change in Keycloak's flow semantics would not be caught by the current suite (Section 11).

Review findings. The review behind this paper found eight issues (Section 10.4). The two most actionable:

- An expired invitation returns a different status code from an unknown address, which is a one-shot way to tell which addresses were invited. Confirmed by experiment.
- The single-use link is a `GET`, so corporate mail scanners that prefetch links can use it up before the person clicks.

Both have small, well-understood fixes.

Recommended next steps.

1. Add a behavioural end-to-end test that performs a real SAML login against the bundled mock IdP.
2. Replace the `GET` redemption with a confirmation page that submits a `POST`.
3. Check the nonce before the expiry so that the two failure pages cannot be told apart.

1 Introduction

1.1 Federation is default-open

Single sign-on through a corporate identity provider is the expected way to reach internal software. Keycloak supports it through **identity brokering**: the realm trusts an upstream SAML or OIDC identity provider (IdP). When a person arrives from that IdP for the first time, Keycloak runs a *first-broker-login* flow that, in its stock form, creates a local account for them on the spot.

That convenience is also a policy decision, and usually the wrong one. Federation answers *authentication*: this person really is `priya@corp.example`. It does not answer *admission*: should this person have an account in this application at all? With the stock flow, admission follows from authentication, so anyone the directory knows can walk in. Roles inside the application can limit what they do, but they already have an account, appear in user lists and receive the realm's default roles. Every common fix has a cost: pre-creating accounts by hand, maintaining a Java plugin inside Keycloak, adopting a third-party extension, or adopting Keycloak's Organizations model (Section 3.3).

1.2 What this project contributes

`keycloak-invite-gate` makes an *invitation* the precondition for federated sign-in, using only facilities Keycloak already has. It contributes a gate with no server-side code (Section 4), an invitation lifecycle with

explicit, tested rules (Section 6), self-repairing provisioning of everything the gate needs inside Keycloak (Section 7), and a written threat model with stated gaps (Section 10).

1.3 Status and scope

The project describes itself as a *focused, tested reference implementation, not a packaged product*. It runs locally with one command (`make up`), and its CI workflow runs the unit and end-to-end suites. There are no production usage figures in the repository, so this paper makes no claims about deployments, adoption or business impact. The value argument in Section 13 rests on design properties that the code and tests support. The paper describes commit `18d9014` (August 2026), which targets Keycloak 26.5.x. File paths are relative to the repository root.

1.4 Evidence conventions

Every claim in a security-relevant design should be traceable to something stronger than its author's confidence. Claims about behaviour carry one of these tags:

Tag	Meaning
[Code]	Established by reading the source at the commit above.
[Unit]	Asserted by a passing unit test (`go test -race ./...`).
[E2E]	Asserted by the end-to-end suite, which the CI workflow runs against a real Keycloak 26.5.4. It needs Docker and was *not* re-run for this paper.
[KC-docs]	Stated in the official Keycloak documentation.
[KC-src]	Confirmed by reading Keycloak's Java source at tag 26.5.0.
[Exp]	Demonstrated by a throwaway experiment run while writing this paper.
[Unverified]	A reasoned expectation that has not been tested.

1.5 How to read this paper

Sections 2 and 3 set up the Keycloak concepts and the problem. Section 4 is the core idea; read it even if you read nothing else. Sections 5–9 explain how the service is built. Sections 10 and 11 test the claims: the threat model, an independent review, and what the tests do and do not prove. Sections 12 to 14 cover operations, value and the path forward.

2 Background

This section introduces just enough Keycloak to make the mechanism in Section 4 obvious rather than clever.

2.1 Realms, identity providers and brokering

A Keycloak **realm** is an isolated namespace of users, clients, roles and authentication flows. A realm can register external **identity providers**. When a person signs in through one, Keycloak acts as a **broker**: it sends the browser to the IdP with a SAML `AuthnRequest`, the IdP authenticates the person, and a signed **assertion** comes back to the realm's broker endpoint, `/realms/{realm}/broker/{alias}/endpoint`. Keycloak must then decide which local user that assertion belongs to. An application can skip Keycloak's login page and go straight to one IdP by adding `kc_idp_hint={alias}` to its authorisation request. The service uses this so that the step after activation is a single click.

2.2 The first-broker-login flow

If the assertion's identity is already linked to a local user, Keycloak signs that user in. If not, Keycloak runs the IdP's **first-broker-login flow**, an ordinary authentication flow whose job is to find or create the local account (Figure 1). The stock flow can review the profile, *create the user if the username and email are unique*, or ask the person to confirm linking to an existing account. A flow is made of **executions**. Each execution is one authenticator, identified by a provider id such as `idp-create-user-if-unique`, plus a requirement setting.

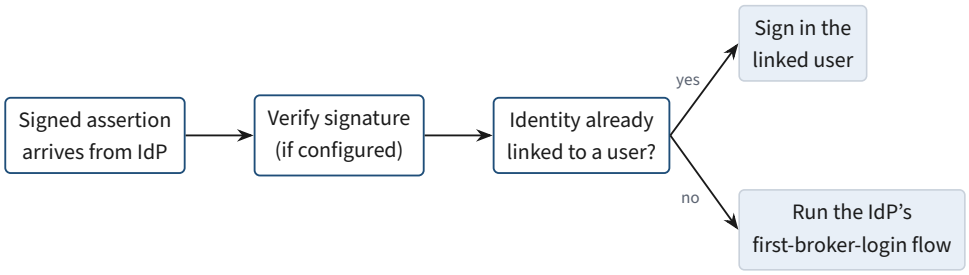


Figure 1. Where the first-broker-login flow sits in a brokered sign-in. The gate lives entirely in the lower branch.

2.3 Execution requirements

Every execution carries one of four requirement values, and the mechanism depends on how two of them combine:

Requirement	Meaning within its flow
REQUIRED	Must run and succeed for the flow to succeed.
ALTERNATIVE	At least one alternative must succeed, *but only when the flow has no REQUIRED (or CONDITIONAL) elements*.
CONDITIONAL	A sub-flow that runs if its conditions hold; it counts as required.
DISABLED	Never runs.

> **The rule the gate rests on**
>
> "Because the Required flow elements are sufficient to mark a flow as successful, any Alternative flow element within a flow containing Required flow elements will not execute." [KC-docs]
> In the engine, `DefaultAuthenticationFlow` removes the alternatives and logs *"REQUIRED and ALTERNATIVE elements at same level! Those alternative executions will be ignored"*. It treats CONDITIONAL sub-flows as required for this purpose too. [KC-src]

2.4 The Admin REST API and the declarative user profile

Everything the service does inside Keycloak goes through the **Admin REST API**. Two of that API's behaviours shape the design.

The user profile is mandatory. Since Keycloak 24 the declarative **user profile** is always on, and attributes it does not declare are *unmanaged*: by default, for new realms, they are "completely ignored by Keycloak" [KC-docs]. (Realms upgraded from a version with the user profile off have unmanaged attributes switched on.) The project's code records the practical consequence: the write succeeds without error and the attribute silently disappears, on create and on update [Code]. An invitation whose nonce hash vanished this way cannot be redeemed, and the only symptom is a "link already used" page on the first click. A realm-wide *unmanaged attribute policy* could loosen this. The service instead declares exactly its three attributes, with admin-only permissions (Section 7.3).

Are search responses partial? The project's code assumes the user search endpoint omits custom attributes, so it follows every search with a fetch by id [Code]. Keycloak 26.5.0's source suggests otherwise: search results are built from the same user-profile representation as a fetch, declared attributes included [KC-src]. The code is also inconsistent, since `ListInvites` reads attributes straight from a search. Neither path is tested against a live server, and the redundant fetch is the source of finding F2.

2.5 SAML assertion signatures

A SAML assertion reaches Keycloak *through the browser*, as a form the IdP has the browser post. The browser is the attacker's territory, so the only thing that makes an assertion trustworthy is its XML signature, checked against the IdP's certificate. A broker configured without signature validation lets anyone who can send an HTTP POST claim any identity. Here that would be fatal: the attacker would simply claim an address that *was* invited. Section 7.4 shows how the service makes that configuration unreachable.

3 Problem Statement and Requirements

3.1 Problem

Given a Keycloak realm federated to a corporate IdP, allow a federated identity to sign in only if an administrator explicitly invited that identity's email address and the invitation was accepted within its validity window. Do this without modifying, extending or redeploying Keycloak, and without introducing a second store of user state.

3.2 Requirements

Table 1 lists the requirements the design answers to, with the section that shows how each is met. Two of them, **R2** and **R3**, are what separate this project from most alternatives.

	Requirement	Why it matters	Met in
R1	Uninvited federated identities are refused.	The point of the project.	§4
R2	No code runs inside Keycloak (no SPI, JAR, theme).	Upgrades stay a configuration concern, not a build.	§4
R3	No separate database.	One system of record, so nothing to keep in sync.	§5
R4	Invitations are unguessable, single-use, short-lived and never stored in clear.	The link travels by email, an untrusted channel.	§6
R5	Only authorised administrators can issue invitations.	Issuing creates users and sends mail from your domain.	§8
R6	The public endpoint does not reveal who was invited.	Invitation lists are organisational data.	§8, §10.4
R7	The identity provider can never run without signature validation.	An unsigned assertion defeats the gate entirely.	§7.4
R8	Provisioning is idempotent and repairs requirement drift.	It runs on every start; a drifted requirement value is an open door.	§7.3
R9	Misconfiguration fails at boot, not at the first invitation.	Shortens the feedback loop; avoids half-working deployments.	§12

Table 1. Requirements and where the paper shows each one is met.

3.3 The design space

There were five realistic ways to solve the problem. Table 2 compares them on the dimensions that matter, using the alternatives' own documentation.

Approach	Gates federated SSO?	Code in Keycloak?	Invitation state	Main cost
Pre-create users by hand; set "create user if unique" to DISABLED	yes	no	Keycloak	Manual work at every hire and departure; no expiry, no email flow.
Custom first-broker authenticator (Java SPI)	yes	yes	Your code	Build and redeploy a JAR against internal APIs on every upgrade.
Registration-invite SPIs (e.g. `keycloak-registration-with-invite-code`)	no	yes	Keycloak	Gates the *local registration form* only, not brokered login.
`keycloak-orgs` (Phase Two)	yes	yes	Extension tables	An extension and its data model; Elastic License v2.
Keycloak 26 Organizations	within the org model	no	Keycloak	Invitations target registration or existing users; members of an org-linked IdP join automatically.
This project	**yes**	**no**	**Keycloak user attributes**	Depends on flow semantics (pinned by tests); a small external service.

Table 2. The design space. Facts about third-party projects are from their repositories and the Keycloak 26 documentation, checked in October 2026.

Three observations narrow the field. **Invitation SPIs solve a different problem**: they gate Keycloak's own registration form, which a federated identity never sees. **Organizations is first-party but has its own model**: an invitation either asks an existing user to confirm membership or sends a new user through the realm's *self-registration* flow, and users of an IdP linked to an organisation are "automatically added as a member". Neither makes a federated login depend on a prior invitation, though if the model fits your tenancy you should use it (the project's README says the same). **Anything that ships Java into Keycloak trades one risk for another**: the flow semantics this project depends on are a documented part of Keycloak's configuration surface, while a custom authenticator depends on internal APIs.

The chosen design is the only one that meets **R1**, **R2** and **R3** together. What it costs is a dependency on Keycloak's flow-execution semantics, and Section 11 looks closely at how well that dependency is pinned.

4 The Mechanism: Two Proofs, No Extension

The whole design comes down to two decisions, each of which is ordinary Keycloak configuration. Everything else in the repository supports these two.

4.1 The invitation is a disabled user

Issuing an invitation creates a Keycloak user with `enabled=false`. The username and email are both the invited address, and three attributes hold the invitation state: the hex SHA-256 of a one-time nonce, an expiry timestamp and a status of `pending`. There is no invitation table anywhere. The invitation *is* the user record before it is activated [Code: `CreateShadowUser`].

Two properties of that record do the gating:

- **It exists**, so a first-broker-login flow that only links to existing users has something to link to. An address that was never invited has no record.
- **It is disabled**, so even a successful link cannot become a session until the invitation is accepted. Keycloak refuses a brokered login to a disabled user with `USER_DISABLED` [KC-src].

4.2 A first-broker-login flow that cannot create users

At boot the service creates its own top-level flow, `invite-gated-first-broker-login`, and binds the identity provider to it [Code: `ensureBrokerFlow`, `ConfigureSAMLIdP`]:

```
var brokerFlowSteps = []struct{ provider, requirement string }{
    // Find the shadow user the invitation created. No user, no login.
    {"idp-detect-existing-broker-user", "REQUIRED"},
    // Bind the arriving federated identity to it, with no user-facing prompt.
    {"idp-auto-link", "REQUIRED"},
    // Retained and inert: REQUIRED steps in the same flow mean ALTERNATIVEs never run.
    {"idp-create-user-if-unique", "ALTERNATIVE"},
}
```

Listing 1. The gate, as the service declares it (`internal/keycloak/bootstrap.go`).

Combined with the rule in Section 2.3, the outcome for an arriving identity is fully determined. Figure 2 traces every path.

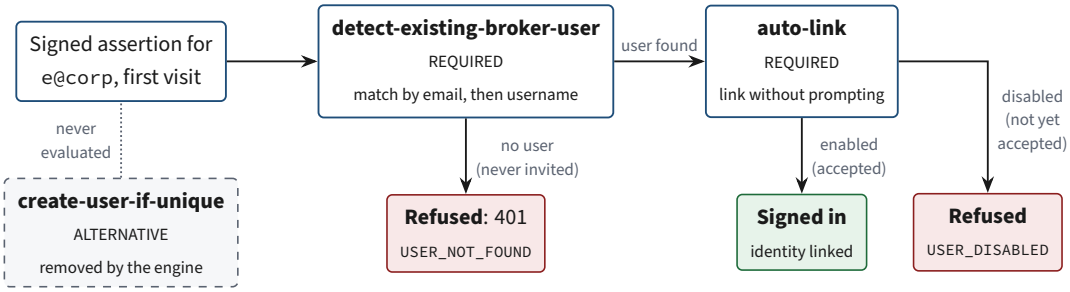


Figure 2. Every path through the invite-gated flow. Error names are Keycloak's own [KC-src]. The ALTERNATIVE step is discarded because REQUIRED steps share its flow.

Why keep an inert step? `idp-create-user-if-unique` never runs as configured. It stays in the flow on purpose. It records, inside Keycloak where an operator will look, exactly which switch turns the gate off, and the end-to-end suite asserts that it is *not* REQUIRED [E2E]. Deleting it would make the flow shorter but harder for the next operator to read.

Why a dedicated flow? Editing the built-in `first broker login` flow would change behaviour for every identity provider in the realm. A separate flow, bound to one IdP, keeps the blast radius equal to the intent [Code].

4.3 Relationship to Keycloak's documented recipe

The two REQUIRED steps are not a trick the project discovered. The Keycloak Server Administration Guide describes a "detect existing user first login flow" made of exactly *Detect Existing Broker User* and *Automatically Set Existing User*, both REQUIRED, whose stated purpose is that "only users already registered in this realm can log in" [KC-docs]. When no user matches, the authenticator renders a 401 page ("User {0} authenticated with identity provider {1} does not exist. Please contact your administrator.") and records the event error `USER_NOT_FOUND` [KC-src].

> **What the project actually adds**
>
> Keycloak supplies the *refusal*: a flow that admits only pre-registered users. What it does not supply is a safe way to decide *who gets pre-registered*. The project's contribution is that decision: an authorised, auditable, expiring, single-use invitation that pre-registers a disabled account and activates it only when the invitee responds. The gate therefore rests on a documented, supported configuration rather than an accident of implementation. That is the strongest argument for its durability.

4.4 Two proofs, one activation

Under these rules, a federated sign-in for address e succeeds only if

$$\underbrace{\mathsf{SignedAssertion}(e)}_{\text{IdP: you are } e} \wedge \underbrace{\exists u : u.\mathsf{email} = e}_{\text{admin: } e \text{ was invited}} \wedge \underbrace{u.\mathsf{enabled}}_{\text{invitation accepted in time}}$$

and, for a shadow user, `enabled` becomes true in exactly one place: the accept transition, after the nonce is verified before expiry [Code]. (A Keycloak administrator can still enable a user by hand; that path is outside the service's control by definition.)

The README describes this as needing *two independent proofs*. It helps to be precise about what each one proves:

- **Proof of intent:** the invitation record. It exists only because an authenticated administrator holding the `invite-admin` role asked for it.
- **Proof of identity:** the IdP's signed assertion. It is the only thing that authenticates the person.
- **The click is neither.** Redeeming the nonce shows that someone with the link acted within the window. It is an *activation* step: it limits how long pre-provisioned access can sit unclaimed, and it confirms the mail reached someone. It is deliberately not a credential. A leaked link can enable an account early, but it cannot sign anyone in (Section 10, T10).

Neither proof is enough on its own. An attacker who intercepts the email still has to pass the IdP as that exact address. A directory member who passes the IdP still has to have been invited.

4.5 Truth table

Invited?	Accepted?	Valid signed assertion for e?	Outcome	Enforced by
no	–	yes	Refused (401)	detect-existing finds no user [KC-src]
yes	no	yes	Refused	user disabled [KC-src]
yes	yes	yes	**Signed in**; identity linked	both REQUIRED steps succeed
any	any	no (unsigned or forged)	Refused	signature validation [Code, KC-docs]
yes	yes	yes, later visits	Signed in via the linked identity	already-linked path; a later `enabled=false` still blocks (`ACCOUNT_DISABLED`) [KC-src]

4.6 What the gate does and does not cover

The gate covers *first* brokered logins through *the IdP bound to this flow*. Several other ways into a realm are outside it, and an operator needs to know them:

- **Other identity providers** in the same realm keep whatever first-broker-login flow they are bound to. A social login added later with the default flow would be a second, open door.
- **Self-registration.** A realm the service creates leaves registration disabled, which is Keycloak's default. The service does not enforce this on a realm that already exists.
- **Users created by other means**, such as directory sync or manual admin work, are not invitations. The service refuses to issue an invitation for an active account like that (Section 6.5).
- **After the first link**, admission is no longer the gate's job. Revoking access means disabling or deleting the user in Keycloak.

5 System Architecture

5.1 Shape

`invite-gate` is a single stateless Go binary. It talks to three things: Keycloak's Admin REST API (for the realm, the users, the flow and the IdP), Keycloak's token introspection endpoint (to authorise administrators), and one mail provider. Figure 3 shows the components and their dependencies.

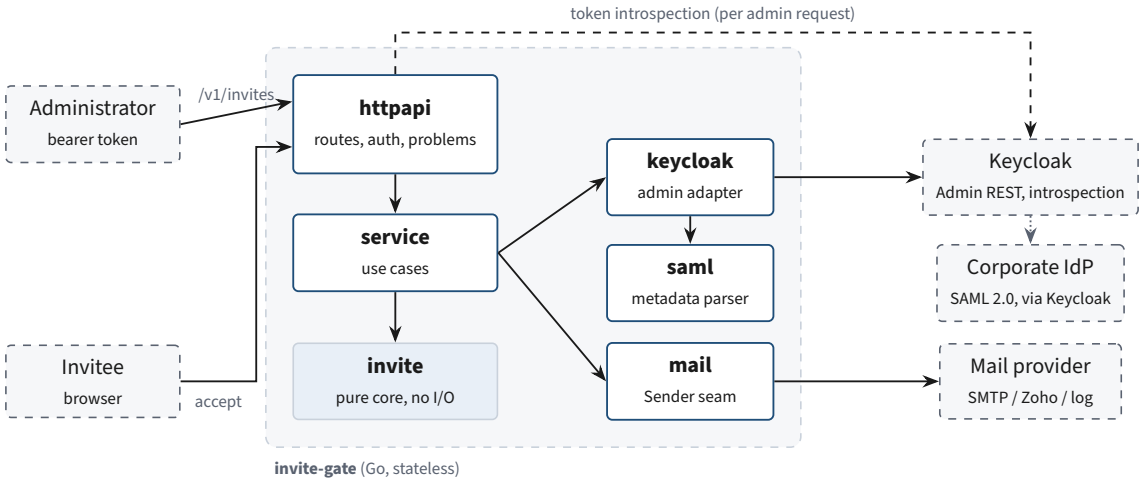


Figure 3. Components and call paths, from caller to callee. The `invite` core is used by the service and, for its types, by the Keycloak adapter. It depends on nothing.

5.2 Layers and the dependency rule

Dependencies point one way: `httpapi` → service → (`keycloak` | `mail`), and `keycloak` → `saml`. The pure `invite` core sits beneath the service and the adapter and depends on nothing. Table 3 lists the packages.

Package	Responsibility	I/O	Lines
`internal/invite`	Nonce minting and verification, expiry, the re-invite decision table	none; clock injected	166
`internal/saml`	Parse IdP metadata; refuse metadata without a signing certificate	none	110
`internal/config`	Read and validate the environment once, reporting every problem together	env	192
`internal/mail`	`Sender` interface; SMTP, Zoho and log providers; message rendering	mail provider	437
`internal/keycloak`	Admin REST client; shadow-user operations; idempotent provisioning	Keycloak	834
`internal/httpapi`	Use cases (`Service`), handlers, admin auth, problem responses	orchestrates	662
`cmd/invite-gate`	Wiring, provisioning order, graceful shutdown	–	159
Total production Go			2,560

Table 3. Packages, what they do and their size (non-test lines) [Code].

5.3 Functional core, imperative shell

The security-critical rules live in the `invite` package: how a nonce is minted and verified, when an invitation counts as expired, and what a repeat invitation does. All of it is pure functions over plain values, and the

current time is passed in as an argument. That choice is what makes the rules *exhaustively* testable. The decision table is a table-driven test, the expiry boundary is tested to the second, and none of it needs Keycloak, a mock or a sleep, because there is nothing to mock [Unit].

The packages around the core are deliberately thin. `Service.Invite` reads the current state, asks `invite.Decide` what to do, and carries out the answer. Apart from the domain allow-list and the revoke guard, it holds no rules of its own about *whether* to act.

5.4 Two seams

The service depends on two interfaces rather than on concrete clients:

```
type Directory interface {
    FindUserByEmail(ctx context.Context, email string) (*keycloak.User, error)
    CreateShadowUser(ctx context.Context, email string, s invite.Secret) (string, error)
    RefreshInviteSecret(ctx context.Context, id string, s invite.Secret) error
    AcceptInvite(ctx context.Context, id string) error
    DeleteUser(ctx context.Context, id string) error
    ListInvites(ctx context.Context) ([]keycloak.User, error)
    AssignRealmRole(ctx context.Context, userID, role string) error
}
```

Listing 2. The `Directory` seam (`internal/httpapi/service.go`).

`Directory` lets the HTTP tests drive the full request path (auth, decision mapping, enumeration resistance) against an in-memory fake, while the real Keycloak client is exercised only by the end-to-end suite. No test checks a mock pretending to be Keycloak. It is also where a database-backed invitation store would plug in, with the gate unchanged. `mail.Sender` has just `Send` and `Name`; adding a provider means writing one small type (Section 9).

5.5 No database; standard library only

Apart from two cached access tokens (Keycloak admin and Zoho), the service keeps no state. Invitation state is three user attributes plus the built-in `enabled` flag, so Keycloak is the single source of truth. There is no consistency problem between an invitation store and the directory, because there is only one store, and any number of replicas can run behind a load balancer. The price is that listing is a realm-wide attribute query and burning the secret also destroys the evidence that an invitation existed. The project lists both as known gaps (Section 10.5) rather than building a store on speculation.

`go.mod` has no `require` block: SAML parsing uses `encoding/xml`, mail uses `net/smtp` and `mime/multipart`, routing uses Go 1.22+ method-and-pattern `ServeMux` routes, and logging uses `log/slog`. For a component that holds realm-admin credentials, a zero third-party dependency graph removes a whole class of supply-chain risk and keeps the audit scope at 2,560 lines.

5.6 Boot sequence

`cmd/invite-gate/main.go` runs a fixed sequence, and the order matters [Code]:

1. **Load and validate configuration.** Every problem is collected and reported in one error, so a bad deployment fails once, not one restart per mistake.
2. Build the structured JSON logger, the Keycloak client and the selected mail sender.
3. **Provision Keycloak** (realm, user profile, broker flow) within a 60-second budget, *before* the listener opens. A service that accepted invitations against an unprepared realm would issue invitations it cannot store.
4. Log the service-provider values to register with the IdP (ACS URL and entity id).
5. If `IDP_METADATA_FILE` is set, parse it and provision the SAML IdP. If not, warn loudly that nobody can

complete SSO yet.

6. Serve HTTP with explicit timeouts (header 10 s, read and write 30 s, idle 60 s), and shut down gracefully on `SIGINT` or `SIGTERM`, with a 15-second drain.

6 The Invitation Lifecycle

6.1 Data model

An invitation has no table of its own. It is a Keycloak user in a particular state, described by three custom attributes and the built-in `enabled` flag:

Attribute	Meaning	After acceptance
`invite_status`	`pending` or `validated`	`validated`
`invite_nonce_hash`	Hex SHA-256 of the raw nonce bytes	deleted (“burned”)
`invite_expires_at`	Unix seconds; the expiry second itself is still valid	deleted
`enabled` (built-in)	`false` until accepted. This is what the gate reads	`true`

All three attributes are declared in the realm’s user profile with view and edit permission for `admin` only, so an invitee can never read their own nonce hash through the account console, or change it [Code].

6.2 State machine

Figure 4 shows the states and the transitions between them. Note that a wrong nonce changes nothing, an expired invitation is cleaned up lazily, and nothing can take an active account back to pending.

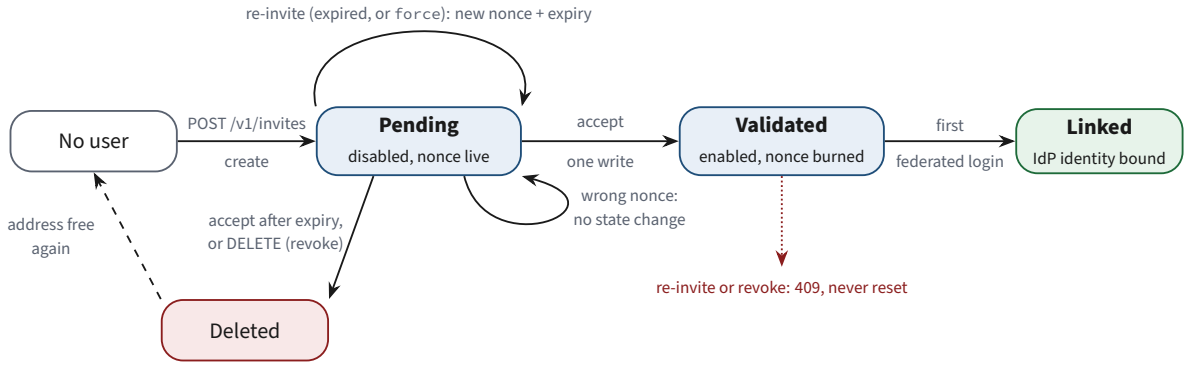


Figure 4. The invitation state machine [Code]. Unit tests cover create, refresh, accept and the wrong-nonce loop. Expiry cleanup and revocation are untested.

6.3 Minting the secret

`invite.NewSecret` reads 32 bytes from `crypto/rand` and returns a `Secret` with three fields. The hex nonce goes into the email link. The hex SHA-256 of the raw bytes and the expiry (`now + TTL`, 15 minutes by default) are stored. The raw nonce is returned to the caller once and is never persisted or logged [Code, Unit].

Why a plain hash, not a password KDF? Password hashing functions such as bcrypt, scrypt and Argon2 exist because passwords have little entropy and an attacker who steals the hashes can work through a

dictionary. A 256-bit uniformly random nonce has no dictionary. Recovering it from its hash is a 2^{256} preimage search, so a work factor would cost latency and buy nothing. For the same reason, online guessing is not a real threat: an attacker sending a billion guesses a second for the whole 15-minute window tries about 9×10^{11} of the $2^{256} \approx 1.2 \times 10^{77}$ possible values.

6.4 Verification and the order of its checks

```go
func Verify(r Record, nonce string, now time.Time) error {
    if !r.hasLiveSecret() { // status pending AND hash present AND expiry present
        return ErrNotPending
    }
    if r.IsExpired(now) { // now > expiresAt; a zero expiry counts as expired
        return ErrExpired
    }
    raw, err := hex.DecodeString(nonce)
    if err != nil {
        return ErrNonceMismatch // malformed input is indistinguishable from a wrong guess
    }
    sum := sha256.Sum256(raw)
    got := hex.EncodeToString(sum[:])
    if subtle.ConstantTimeCompare([]byte(got), []byte(r.NonceHash)) != 1 {
        return ErrNonceMismatch
    }
    return nil
}
```

Listing 3. `invite.Verify`, slightly condensed (`internal/invite/invite.go`).

Each check reflects a decision:

- **Fail closed on partial records.** A record missing its hash or expiry is not live, and a record with no expiry counts as *expired*, never as permanent. A half-written or tampered record cannot become an invitation that never expires [Unit].
- **The expiry boundary is inclusive**: valid at `expiresAt`, invalid one second later. Both edges are tested [Unit].
- **Malformed input collapses into a mismatch**, so a caller cannot tell "not hex" from "wrong" [Unit].
- **Constant-time comparison is defence in depth.** The operands are hashes, and learning a prefix of a hash by timing does not reveal the preimage. The project uses the constant-time compare anyway, because it costs nothing and leaves nothing to argue about.
- **Expiry is checked before the nonce.** The stated reason is that a correct and an incorrect nonce for an expired invitation should look the same. Section 10.4 (finding F1) argues that this ordering protects against a threat that does not exist and opens a small one that does.

6.5 Re-invitation: an explicit decision table

Re-invitation is idempotent by email address but never silent. `invite.Decide` maps the existing state and the `force` flag to an action:

Existing state for the address	Default	With `force=true`
No user	create	create
Pending, not expired	**409** `invite_already_active`	refresh nonce and expiry
Pending, expired	refresh nonce and expiry	refresh nonce and expiry
Validated, or any account without a live invitation	**409** `user_already_active`	**409** `user_already_active`

Two rows encode judgement rather than mechanics:

- **A live invitation is not replaced silently.** Re-issuing it would invalidate a link the invitee might be about to click, and they would get an unexplained failure. `force` makes "I know, resend anyway" explicit, and the test suite checks that the old link stops working afterwards [Unit].
- **An active account is never reset, even with force.** A user with no live invitation might have been validated, created by directory sync, or deliberately disabled by an administrator. Re-inviting it would make the admin endpoint a way to re-enable an account (an account-takeover primitive), so the core refuses it outright [Code; Unit for validated and pre-existing enabled accounts].

6.6 Acceptance: one write, then verify

When the nonce verifies, `AcceptInvite` sends *one* `PUT` that sets `enabled=true` and `emailVerified=true`, sets the status to `validated` and clears both secret attributes. Keycloak merges attribute maps, so absent keys would survive. The keys are cleared by sending them as empty lists [Code]. The service then reads the user back and refuses to report success if the hash is still there: "the burn is a security property, not a nicety" [Code, E2E].

Why one write? Consider a crash between two separate writes:

Ordering	State left behind by a crash between the steps
Enable, then burn	An **enabled account with a live nonce**: the link still works, so single use is broken.
Burn, then enable	A **burned, disabled account**: the invitee can never activate it, and a re-invite is refused because there is no live secret.
One update	Either nothing changed or everything did.

After success the browser is redirected (302) to `APP_LOGIN_URL` with three query parameters: `activated=true`, the email address, and `kc_idp_hint=corp`. The application is expected to pass the hint to Keycloak, which then goes straight to the IdP, so the step after activation is a single click.

6.7 Expiry, revocation and cleanup

Expiry is discovered lazily, on the next accept attempt for that address, and the shadow user is deleted then so that a fresh invitation is not blocked by an email conflict. This is safe because the account was never enabled or linked. A background reaper would be tidier; the project chose the simpler trade and says so. **A wrong nonce never mutates state**: only successful verification and expiry cleanup write. Otherwise anyone who knew an invitee's address could destroy their invitation by guessing badly (T6), and a unit test sends five bad guesses and then requires the real link to work [Unit]. **Revocation** (`DELETE /v1/invites/{email}`) deletes the user unless it is validated or enabled, which returns 409 and sends the administrator to Keycloak [Code]. That guard is looser than the decision table: a disabled account that was never an invitation is deleted too (finding F8).

6.8 End to end

Figure 5 puts the pieces together, from the administrator's request to the invitee's first signed-in page.

7 Keycloak Integration

`internal/keycloak` is the largest package (834 lines) and the only one that talks to Keycloak's Admin API. Much of its size is knowledge: each API pitfall it handles would otherwise show up as an invitation that mysteriously "was already used".

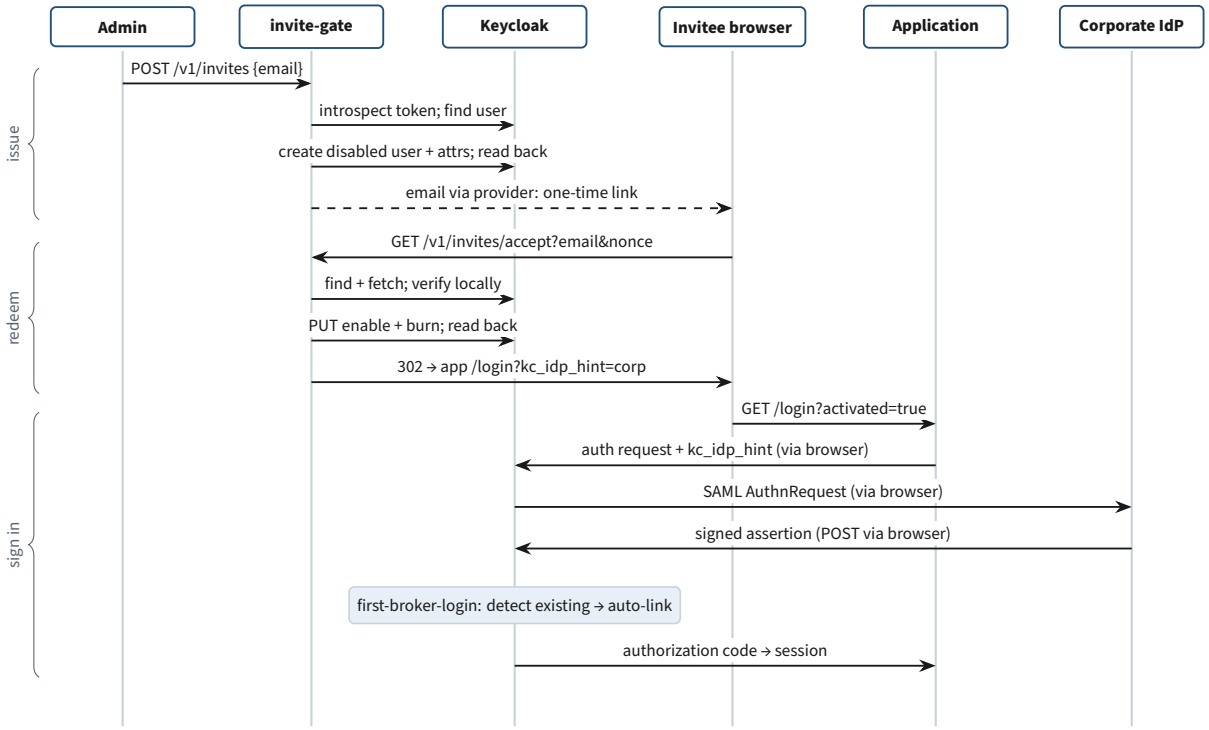


Figure 5. Issue, redeem and sign in. Steps up to the 302 are this service. Everything after is standard Keycloak brokering, gated by the flow in Figure 2.

7.1 The admin client

The client gets an admin token with the password grant against the master realm's `admin-cli` client (a known least-privilege gap, Section 10.5). It caches the token and retires it 30 seconds early so that a request in flight cannot outlive it. The cache mutex is held during a refresh, so concurrent callers wait for one refresh rather than stampeding the token endpoint. A 401 invalidates the cache and retries once, because admin tokens are short-lived and the first request after a quiet spell would otherwise fail. (One inconsistency: `createdID`, which reads a new user's id from the `Location` header, does not share this retry.) `ErrNotFound` is a sentinel that separates "no such user" from "the request failed", since conflating them would turn an outage into a silent "you were never invited". Error messages carry the status and at most 512 bytes of body; a failed admin login reports only the status [Code].

7.2 Pitfalls encoded in the adapter

Table 4 lists the Keycloak behaviours the adapter defends against. Every one of them fails silently if ignored.

The project calls the user-profile requirement "the single least obvious requirement in the project". It is the clearest example of why the adapter reads back what it writes. Writing state that silently fails to persist is worse than failing loudly, because the error appears much later, in front of a different person, as a misleading message.

7.3 Idempotent provisioning

`Bootstrap` runs on every start, so each step checks before it writes. Running it twice is a test against a real Keycloak [E2E].

Keycloak behaviour	Symptom if ignored	Countermeasure
User profile (24+) ignores undeclared attributes	Every link reports “already used” on the first click; no error anywhere	Declare the three attributes at boot; read back after every write; dedicated E2E regression test
User search omits custom attributes (assumed; see §2.4)	Every invitation looks burned	Search, then fetch by id (`FindUserByEmail`)
Attributes sent on create are not reliably kept (observed by the project)	Hash missing from a fresh invitation	Write attributes again with an explicit `PUT`, read back, and delete the half-built user if that fails
Attribute maps are merged on update	“Burned” keys survive	Clear keys with empty lists; verify the burn by reading back
New flow executions start as DISABLED [KC-src]	Gate configured but inactive	Reconcile requirement values on every boot
Brokered identities match on email *or* username	One person becomes two accounts if the username is truncated	Username is the full email address

Table 4. Silent-failure modes of the Keycloak Admin API and how the adapter handles each one [Code unless marked].

Step	Checks	Writes when	Repairs drift?
`ensureRealm`	`GET` the realm	404: create it, enabled	–
`EnsureUserProfileAttributes`	names present in the profile	any missing: add with admin-only view and edit	re-adds missing declarations
`ensureBrokerFlow`	flow alias exists	absent: create a top-level `basic-flow` and add the three executions	**requirement values**, every boot
`ConfigureSAMLIdP`*	IdP alias exists	always: `PUT` the full representation (or `POST` if new)	signature settings and flow binding
`ensureEmailMapper`*	mapper named `email`	absent: create	presence only

*Only when `IDP_METADATA_FILE` is set.

The reconcile loop matters more than it looks. Keycloak adds new executions as DISABLED [KC-src], so *the loop is the code that switches the gate on*, even on first creation. It also runs on every later start: if an operator changes `idp-auto-link` to ALTERNATIVE in the console, the next restart puts it back. In the code’s own words, “the gate is the requirement values, so drift here is a silently open door.” Section 10.4 (F7) covers the drift the loop does *not* repair.

7.4 The identity provider: signature validation is not optional

`saml.Parse` reads the IdP’s metadata with `encoding/xml`. It extracts the entity id, the single sign-on URL (preferring the HTTP-Redirect binding and falling back to HTTP-POST) and a signing certificate. A `KeyDescriptor` with no `use` attribute counts as a signing key; one marked `encryption` does not. Most importantly, metadata with no signing certificate is **rejected with an error, not downgraded to a warning** [Unit, 100% statement coverage]. Because `ConfigureSAMLIdP` only runs after a successful parse, Keycloak can never be configured to accept unsigned assertions from this IdP. One inconsistency: the parser prefers the Redirect endpoint to avoid a form interstitial, but the IdP is provisioned with `postBindingAuthnRequest=true`, so Keycloak POSTs to that endpoint anyway. An IdP that serves the two bindings at different URLs may reject it [Unverified].

Setting	Value	Reason
`validateSignature`	`true`	Without it anyone can assert an invited address (T9)
`wantAssertionsSigned`	`true`	The assertion itself must be signed, not only the response
`signingCertificate`	from metadata	The parser guarantees one is present
`firstBrokerLoginFlowAlias`	the gated flow	This line binds the gate to the IdP
`nameIDPolicyFormat`	`emailAddress`	Brokered username equals the shadow user's username
`trustEmail`	`true`	The directory owns the address; no second verification prompt
`storeToken`	`false`	The upstream assertion is not needed after login
`allowCreate`	`true`	SAML `NameIDPolicy`: lets the IdP mint a NameID. It has nothing to do with Keycloak creating users
`wantAuthnRequestsSigned`	`false`	Requests are unsigned (possible hardening)

An attribute mapper copies the assertion's `Email` attribute onto the user's `email`, so the arriving identity matches the shadow user. At boot the service also logs the values to register with the IdP: the assertion consumer URL <external>/realms/<realm>/broker/<alias>/endpoint and the SP entity id, which defaults to `keycloak-<realm>`.

7.5 Version notes: Keycloak 23 to 26

The changelog's migration note records two changes since Keycloak 23 that shaped the current code:

- **Keycloak 24 made the user profile always-on**, which produced the silent-attribute-loss problem above and the boot-time declaration that fixes it.
- **Keycloak 26 deprecated** `KEYCLOAK_ADMIN/KEYCLOAK_ADMIN_PASSWORD` in favour of `KC_BOOTSTRAP_ADMIN_USERNAME/_PASSWORD`. The old names still work in 26.5 with a deprecation warning [KC-src], and the compose file and CI use the new ones. (The project's changelog says "renamed". "Deprecated" is the more precise word.)

8 HTTP API and the Administrative Trust Boundary

8.1 Endpoints

The service exposes five routes using Go's method-and-pattern `ServeMux`. Handlers parse input, call the service and map errors to responses. None of the invitation logic lives in them [Code].

Route	Caller	Success	Failures (problem `code`)
`POST /v1/invites`	admin	202	400 `invalid_body`, `invalid_email`; 403 `domain_not_allowed`; 409 `invite_already_active`, `user_already_active`; 500 `internal_error`, including undelivered mail
`GET /v1/invites`	admin	200	500 `internal_error`
`DELETE /v1/invites/{email}`	admin	204	400 `invalid_email`; 404 `not_found`; 409 `user_already_active`; 500
`GET /v1/invites/accept`	invitee	302	400 or 410, as HTML pages
`GET /health`	anyone	200	– (reports the active mail provider)

Every admin route can also return 401 `unauthorized` (no token, or an inactive one), 403 `forbidden` (missing role) and 503 `introspection_unavailable`.

8.2 Authorising administrators

`POST /v1/invites` creates users in your directory and sends mail from your domain. Left open, it is a spam relay and a way to pollute the user store. Rendering an "invite" button only for administrators is not authorisation, so every admin route is wrapped in `requireRole`:

1. Require `Authorization: Bearer <token>`. Otherwise return 401.
2. Send the token to Keycloak's introspection endpoint (RFC 7662), authenticating as a confidential client. Keycloak accepts introspection only from confidential clients [KC-docs].
3. If introspection itself fails, return **503 and stop**. An unverifiable token is never admitted [Unit: "introspection failure does not fall open"].
4. An inactive token gets 401. A token without the configured realm role (`invite-admin` by default) gets 403. Only then does the handler run [Unit].

Why introspection rather than local JWT verification? Checking a JWT's signature locally would save a network round trip, but a token revoked seconds ago would keep working until it expired. With introspection, revocation takes effect immediately. Role removal does too: Keycloak rebuilds the introspection response, including `realm_access.roles`, from the current session and role mappings, rather than returning the token's snapshot [KC-src]. For an endpoint that creates directory users, one extra round trip per admin request is the right trade.

8.3 The public accept endpoint

`GET /v1/invites/accept?email=…&nonce=…` is reached by a person clicking a link in a mail client. It renders HTML rather than JSON and has exactly two failure pages:

- **Expired** (410): "This invitation has expired."
- **Invalid** (400): "This invitation link cannot be used", for every other cause: an unknown address, an invitation already used, a wrong or malformed nonce, or missing parameters.

Collapsing the causes is deliberate (T8). If "never invited" and "already used" looked different, the endpoint would answer "has this address been invited?" for anyone who asked. A unit test requires byte-identical status and body for an unknown address, an already-used link and missing parameters [Unit]. A wrong nonce on a live invitation takes the same code path [Code]. The precise reason goes to the server log instead. Section 10.4 examines two remaining ways to tell the cases apart.

Because the nonce travels in the query string, the endpoint also guards the URL:

- The failure page sends `Referrer-Policy: no-referrer` (and the equivalent `<meta>`), so the browser does not forward the URL to anything the page loads [Unit for the header].
- It sends `Cache-Control: no-store` [Code].
- The request logger records the method, path and status, and *never the raw query* [Code].

8.4 Error model

API errors are `application/problem+json` documents with a numeric `status`, a machine-readable `code` and a human `detail`. The code calls this RFC 7807. That RFC was obsoleted by RFC 9457 in July 2023, which keeps the same media type and format, and `code` is a permitted extension member. A client can switch on `code` without parsing prose. For example, a 409 is either `invite_already_active` (retry with `force`) or `user_already_active` (do not retry).

8.5 Input handling and server hardening

- **Bodies are capped at 4 KiB** with `http.MaxBytesReader` [Code].

- **Addresses are normalised** (trimmed and lower-cased) before they reach the directory. Otherwise the same person invited twice with different capitalisation would become two users [Unit: lower-casing].
- **Email validation is deliberately shallow**: a local part, an `@`, and a domain containing a dot that does not end with one, at most 254 characters, with no whitespace. Keycloak and the mail provider have the final say. A strict regular expression would reject valid addresses and gain nothing [Unit].
- **Domain restriction.** `ALLOWED_DOMAIN` limits invitations to one domain, matched exactly against the text after the last `@` [Code].
- **Default role.** If `DEFAULT_ROLE` is set, it is granted at invitation time. A failure to assign it is logged rather than returned. The invitation is still valid, and failing the request would leave a working shadow user behind an error response [Code].
- **Timeouts** are set on every phase of the server, and admin introspection uses a 10-second client timeout [Code].

9 Email Delivery

9.1 The `Sender` seam and its providers

```go
type Sender interface {
    Send(ctx context.Context, m Message) error // Message: To, Subject, HTML, Text
    Name() string                              // shown in logs and in /health
}
```

The interface is deliberately this small, because every speculative method would make every provider harder to write. Choosing a provider is one variable, `MAIL_PROVIDER`, and no provider detail leaks into the invitation flow.

Provider	Behaviour and safeguards
`smtp` (default)	Standard-library `net/smtp` with a dial timeout. Optional `STARTTLS` verifies the server certificate against the configured host. PLAIN authentication is used when a username is set, and Go refuses to send it unencrypted to anything but localhost. Messages are `multipart/alternative` with the text part first, so clients that take the last part they understand show the HTML [Unit].
`zoho`	Zoho Mail REST API. A refresh token is exchanged for access tokens, cached under a mutex and retired a minute early. Zoho is partitioned by region, and a token from one region's host is rejected by another's, so both base URLs are *required, with no default*, and a missing one fails at boot instead of as a confusing 401 [Unit]; a wrong region still fails at send time. Credentials go in the request body, never the query string, and errors carry Zoho's code, never a secret [Unit].
`log`	Writes the message, link included, to the log instead of sending it. Selecting it also needs `ALLOW_LOG_MAIL_PROVIDER=yes`; without that second opt-in the service will not boot [Unit].

9.2 Rendering, and what happens when delivery fails

Every message has an HTML part and a plain-text part, because some corporate clients strip HTML and a blank invitation fails as surely as a missing one. The HTML uses `html/template`, which escapes the accept URL for its `href` context. A test injects `"><script>` into the URL and checks that it cannot break out [Unit]. The validity window is written in words ("1 minute", "15 minutes", "2 hours") [Unit]. The footer says the link "expires on its own and grants no access by itself", which is literally true under the two-proof design. One gap: the Zoho provider sends only the HTML part, so its recipients lose the plain-text fallback [Code].

Delivery happens *after* the invitation is stored. If the provider fails, the invitation exists but nobody has its link. The service returns 500 rather than claiming success [Unit] and logs "invitation stored but not delivered" [Code]. A plain retry then gets 409 `invite_already_active`. The administrator retries with `force`, which mints a new nonce and makes any link that might have escaped useless. Storing before

sending means a failure leaves a recoverable invitation, never a delivered link to an invitation that does not exist.

10 Security Analysis

The repository ships a threat model (`docs/security-model.md`) that names ten threats, four non-goals and six known gaps. This section summarises that model, adds the trust assumptions it leaves implicit, and reports what an independent review found while this paper was being written.

10.1 Assets and trust boundaries

Asset	Lives in	Impact if lost
Raw nonce	email, mailbox, URL	An attacker can *activate* an account, but cannot sign into it
Nonce hash	Keycloak user attribute	Nothing directly; preimage-resistant
Keycloak admin credentials	service environment	Full compromise of the Keycloak server (master realm)
Mail credentials	service environment	Mail sent as your domain

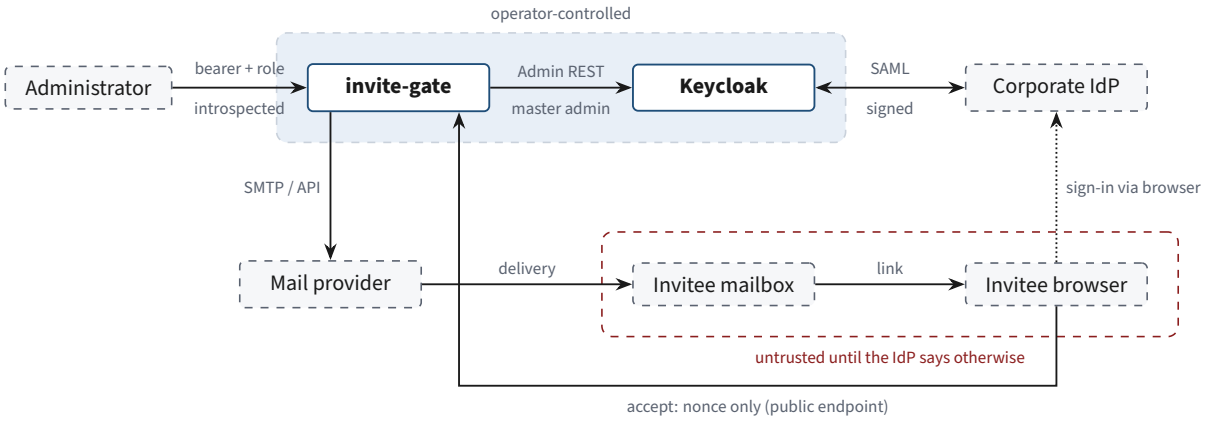


Figure 6. Trust boundaries. The mailbox is a delivery channel, not an authentication factor. The browser carries every SAML message and is assumed hostile.

10.2 Threats and controls

Table 5 lists the ten threats with their controls and the evidence for each.

T10 is the most instructive entry, because it is *accepted* rather than fixed, and the reasoning holds up. The nonce authorises exactly one state transition, from disabled to enabled, and an enabled account still cannot be used without the IdP. A leaked link at worst lets an account become active slightly before its owner clicks.

10.3 Trust assumptions, made explicit

A threat model is only as good as the assumptions under it. These are implicit in the project's documents and worth stating:

A1. The IdP is authoritative for email addresses.
Auto-link binds an arriving identity to the account with the same email or username "without any verification", and Keycloak warns that this "is dangerous in a generic environment where users can register themselves using arbitrary usernames or email addresses" [KC-docs]. The proof of identity is therefore only as strong as the IdP's control over the address it asserts. If directory users can edit their

	Threat	Control	Evidence
T1	Uninvited directory user signs in	Flow links only to existing users; ALTERNATIVE create is discarded	E2E (config); KC-src
T2	Link replay	Nonce burned in the same write that enables the account; read-back check	Unit, E2E
T3	Nonce brute force	256-bit `crypto/rand` nonce; 15-minute window	Unit
T4	Nonce disclosure at rest	Only SHA-256 stored; attribute visible and editable by admin only	Unit, Code
T5	Timing oracle on comparison	Constant-time compare of hashes (defence in depth)	Code
T6	Invitation burned by spraying bad nonces	A mismatch never mutates state	Unit
T7	Abuse of the invite endpoint	Introspected bearer token plus realm role; fails closed	Unit
T8	Email enumeration via accept	Failure causes collapsed into one page	Unit (*see F1, F2*)
T9	Forged IdP assertion	`validateSignature=true`; metadata without a certificate is refused	Unit, Code
T10	Nonce leaks via URL	Accepted: single use, short-lived, not a credential; `no-referrer`, `no-store`, no query logging	Unit, Code

Table 5. The project's threat model, with the evidence behind each control.

own email attribute, an uninvited user could claim an invited address. *Operators should confirm that the asserted address cannot be self-edited, and set `ALLOWED_DOMAIN` to the IdP's own domain.*

A2. The metadata file is authentic.
The signing certificate comes from `IDP_METADATA_FILE`, a local path. If an attacker can replace that file, they choose the certificate the gate trusts. Where the file comes from is the operator's responsibility, and the metadata's own signature is not checked.

A3. Keycloak administrators are trusted.
They can enable users, edit the flow or bind another IdP. The gate is a policy on the federated path, not a control against realm administrators.

A4. The service host is trusted with super-admin rights.
It holds master-realm admin credentials. Compromising the host compromises every realm on the server (known gap 5).

A5. The gated IdP is the only door.
Other IdPs, self-registration and directory sync are outside the gate (Section 4.6).

10.4 Findings from this review

The review behind this paper read every line of the service and checked Keycloak's behaviour against its 26.5.0 source. Throwaway experiments, kept out of the repository, tested two hypotheses. It found eight issues. None lets an uninvited identity sign in. Each one is a gap in a property the project claims, or a deployment hazard.

	Finding	Severity	Evidence
F1	An expired invitation can be told apart from an unknown address	Low	Exp
F2	Response time differs for addresses that have an account	Low	Code
F3	Mail scanners can use up the single-use `GET` link	Medium	vendor docs
F4	Admin tokens are not bound to an audience or client	Low	Code
F5	Read–decide–write without conditional updates	Low	Code
F6	A Keycloak outage is shown to the invitee as an invalid link	Low	Exp
F7	Drift repair covers requirement values, not flow structure or bindings	Medium	Code, KC-src
F8	Revoke deletes any disabled account, not only invitations	Low	Code

F1. Expired invitations are distinguishable (one-shot enumeration)

Observed [Exp]. With a garbage nonce, an address with an expired pending invitation returns **410** and an unknown address returns **400**. A second probe of the same address returns 400, because the first one triggered the cleanup that deletes the shadow user. **Why.** `Verify` checks expiry before the nonce. The stated reason, that a guessed nonce should not be confirmable after expiry, defends against nothing, since a 256-bit nonce cannot be guessed (T3). The failure page assumes "the invitee already holds a link", which the ordering never checks. **Fix.** Check the nonce first, or render expiry as the generic page, and add the expired case to the indistinguishability test.

F2. Timing separates unknown addresses from existing accounts

Observed [Code]. An unknown address costs one Keycloak call (an empty search); an existing account costs two (search, then fetch) before verification. Whether the difference survives network jitter is [Unverified]. It would reveal that an address has an account, not that it was invited. The second call exists only because the code assumes search results lack custom attributes, which Keycloak 26.5.0's source contradicts (§2.4). **Fix.** Drop the extra fetch once a live test confirms search returns the attributes.

F3. Mail scanners can use up the single-use `GET` link

Observed. Microsoft Defender for Office 365 Safe Links scans URLs before delivery and "detonates" some asynchronously, and authentication vendors (Stytch, Supabase) document scanners consuming one-time links. A scanner's `GET` would activate the account and burn the nonce, and the human would then see "This invitation link cannot be used" and never be sent on to sign in. **Impact.** No security impact (T10 holds), but the experience breaks for exactly the corporate audience the project targets. **Fix.** Make `GET` render a confirmation page that `POST`s the nonce. Carrying the nonce in the URL fragment, which browsers never send to servers, would close T10 at the same time.

F4. Introspected tokens are not bound to an audience

Observed [Code]. `requireRole` checks only that the token is active and holds the realm role, so a token issued to *any* client in the realm is accepted, for example one leaked from a less-trusted single-page app, as long as its user holds `invite-admin`. **Fix.** Check `aud`/`azp` against an allow-list, or use a client role.

F5. No conditional writes

Observed [Code]. Issue, accept and revoke read, decide locally and then write, and the Admin API has no compare-and-set on users. Concurrent creates are arbitrated by Keycloak's uniqueness constraint (the loser gets a 500). Two simultaneous valid accepts can both return 302. An accept racing a `force` re-invite can activate the account through the superseded link. **Impact.** No access

beyond the design's, because activation is not access. "Single use" holds for the resulting state, not for the number of successes.

F6. Outages look like broken links

Observed [Exp]. With the directory failing, accept returns 400 and the generic "cannot be used" page, so invitees ask for a new link that fails the same way. **Fix.** Map infrastructure errors to a 503 "try again shortly" page, which reveals nothing about any address.

F7. Drift repair covers requirement values only

Observed [Code, KC-src]. Executions are added only when the flow is created. Later boots correct the requirements of executions that exist, but do not re-add missing ones, remove extra ones, or check other IdPs, and the IdP's flow binding is re-applied only when `IDP_METADATA_FILE` is set. Because Keycloak adds executions as DISABLED, a first boot interrupted part-way through building the flow leaves one that no later boot completes. The E2E suite provisions its own fresh realm, so it cannot see this in a deployed one, and the service would start normally. **Fix.** Reconcile the whole flow structure, and at boot refuse to start if any IdP in the realm can reach a flow that creates users.

F8. Revoke deletes any disabled account

Observed [Code]. `RevokeInvite` refuses only users that are validated or enabled. Any other disabled user is deleted, including an account an administrator disabled on purpose that was never an invitation, which is exactly the account `Decide` treats as untouchable. The guard has no unit test. **Impact.** Admin-only, but destructive and irreversible. **Fix.** Revoke only records whose status is `pending`, and test the refusal.

10.5 Known gaps

The project lists six gaps itself, "because a threat model that claims completeness is not a threat model":

1. No rate limiting on the invite endpoint beyond the role check.
2. The nonce travels in a query string (T10).
3. No audit trail: burning the nonce destroys the evidence that an invitation existed.
4. Listing by attribute query does not scale to very large realms (and is capped at 1,000 results).
5. Admin access uses the password grant against the master realm, where a service account with `manage-users` and `manage-identity-providers` would be least-privilege.
6. An account stays enabled after activation even if its owner never signs in. There is no reaper.

11 Verification

11.1 Strategy: test the logic where it lives

The test design follows from the architecture:

- **The pure core is tested exhaustively**, with an injected clock and no doubles, because it has no dependencies to stand in for.
- **The HTTP layer is tested through its seams.** Tests drive real requests through the real router, auth wrapper, service and core, against an in-memory `Directory`, a capturing `Sender` and a stub introspector. These fakes hold state and behave like the real thing. Tests assert on outcomes and on what the fake directory recorded, never on a mock's expectations of Keycloak's API.

- **Providers are tested on the wire.** The Zoho sender runs against an `httptest` server, which lets the tests inspect real requests, for example to check that no credential reaches a query string.
- **The Keycloak adapter is tested only against a real Keycloak.** There is no mock Keycloak anywhere. A fake that encoded the project's beliefs about Keycloak would pass precisely when those beliefs were wrong.

11.2 The unit suite

Package	Cases	Coverage	What is pinned
`invite`	26	85.3%	Nonce size, hashing, uniqueness, expiry arithmetic; every `Verify` edge; the nine-row decision table
`httpapi`	20	60.5%	Four test functions: the admin gate (five subtests), issuing (six), redeeming (five), health
`saml`	12	100%	Binding preference, certificate rules, refusal of incomplete or unsigned metadata
`mail`	10	53.4%	Rendering and escaping, MIME layout, Zoho token caching and credential handling
`config`	8	97.0%	Aggregated errors, provider requirements, the log opt-in, TTL parsing
`keycloak`	–	E2E only	Covered by the end-to-end suite
Total	**76**		Clean under `go test -race`, `go vet` and `gofmt`

Coverage numbers show where the gaps are. Untested in `httpapi`: the real introspection client; the list and revoke paths, including revoke's guard (finding F8); the expired-accept path (the 410 page and the lazy delete); the `DEFAULT_ROLE` branch; the domain allow-list and its 403; and the directory-error path behind finding F6. `keycloak.User.Record`, the pure function that turns attributes into a record (including the rule that a malformed timestamp counts as expired), is reached only from other packages' tests and the E2E suite, and deserves direct table tests. In `mail`, the SMTP conversation and the log sender are untested.

Tests worth reading. Several tests state a design decision as an assertion: `accepts on the exact expiry second` and `rejects one second past expiry`; `a wrong nonce does not burn the invitation` (five bad guesses, then the real link must work); `failures are indistinguishable across causes` (unknown, used and incomplete requests get one byte-identical response); `force replaces a live invitation and invalidates the old link` (the superseded URL is replayed and must fail); and `introspection failure does not fall open`.

11.3 The end-to-end suite

The E2E suite (`test/e2e`, behind the `e2e` build tag) runs the real adapter against a real Keycloak. It skips cleanly when no server is reachable.

Test	What it pins against a live Keycloak
`TestBootstrapIsIdempotent`	Provisioning twice succeeds both times
`TestUserProfileAttributesPersist`	The nonce hash and expiry survive the write; the shadow user is created disabled (the regression test for the user-profile trap)
`TestAcceptEnablesAndBurns`	After acceptance: enabled, status `validated`, hash and expiry gone
`TestBrokerFlowGatesUninvitedIdentities`	Three subtests: a stranger's address has no user; an unaccepted invitation is disabled; the flow's requirement values read back as REQUIRED / REQUIRED / ALTERNATIVE

11.4 What the end-to-end suite proves, and what it does not

This is the most important caveat in the paper. The project calls `TestBrokerFlowGatesUninvitedIdentities` "the most important test in the repo", and the README says that if a future Keycloak changes how REQUIRED and ALTERNATIVE combine, "that test fails in CI instead of the gate opening in silence". Looking at what the test actually does:

Link in the chain of evidence for T1	Evidence
The flow is configured REQUIRED / REQUIRED / ALTERNATIVE in a live server	E2E
An uninvited address has no user; an unaccepted invitation is disabled	E2E
REQUIRED executions cause ALTERNATIVEs in the same flow to be discarded	KC-docs, KC-src (26.5.0)
Detect-existing refuses when no user matches; disabled users are refused	KC-src (26.5.0)
A real SAML login by an uninvited identity is refused	**not automated**

The suite pins what the provisioning code *produces*: a code change that writes the wrong requirement value, drops a step or creates users enabled would fail it. Because every test first re-runs `Bootstrap` on its own realm, it cannot see drift in a deployed realm. It cannot catch a change in what those values *mean*. Reading back the string "REQUIRED" succeeds whatever the engine does with it. Today, the semantics are established by Keycloak's documentation and by reading its source for this paper, not by an automated test.

> **Closing the gap**
>
> The repository already ships the missing piece of infrastructure: a SimpleSAMLphp mock IdP in `docker-compose.yml`. A behavioural test would provision the IdP from the mock's metadata, then use an HTTP client with a cookie jar to follow the redirects and post the SAML forms, with no browser. It would assert three outcomes: a stranger is refused (Keycloak's `USER_NOT_FOUND` event can be read back through the admin events API), an invited but unaccepted user is refused, and an accepted user receives an authorisation code. Run on every Keycloak upgrade, that test would make the README's claim true.

11.5 Continuous integration

The GitHub Actions workflow has two jobs. **unit** runs a `gofmt` check, `go vet` and `go test -race`. **e2e** starts quay.io/keycloak/keycloak:26.5.4 in `start-dev` mode as a plain container (service containers cannot override the image command), waits on the OIDC discovery document, and runs the tagged suite. Two observations: the workflow triggers on pushes to `main` while the local branch is `master`, so pushes would be tested only through pull requests until one of them is renamed. And the checkout this paper was written from has no remote configured, so this paper cannot confirm a CI run.

11.6 What the demo stack shows

`make up` starts four containers: Keycloak, MailHog, the mock IdP and the service. Out of the box every admin call returns 503, because `Bootstrap` creates neither the confidential introspection client the compose file names nor the `invite-admin` role. Once those and an admin user are created by hand, the stack demonstrates issuing an invitation, catching the mail and redeeming the link through to the redirect. It does not demonstrate federated sign-in:

- `IDP_METADATA_FILE` is not set for the service, so the mock IdP is never registered with Keycloak.
- The redirect target (`localhost:3000/login`) is not part of the stack.
- The mock IdP's assertion consumer URL uses the in-network host `keycloak:8080`, which a browser on the host cannot resolve.

All of these are small fixes, and together they would turn the demo into a complete walk-through.

12 Operations

12.1 Configuration, container and observability

Configuration. Everything comes from environment variables. `.env.example` documents all but `ALLOW_LOG_MAIL_PROVIDER`, and `.env` and tenant metadata files are git-ignored. `config.Load` validates everything at boot and reports *every* problem in one error, because "failing on the first one turns setting up the service into a guessing game" [Unit]. Internal and external Keycloak URLs are separate (the service may reach `keycloak:8080` while browsers need a public host). The code has no default where one would be wrong, such as Zoho's region URLs, and dangerous choices, such as the log provider, need a second key. Appendix A lists every variable.

Container. A two-stage build compiles a static binary in `golang:1.25-alpine` (`CGO_ENABLED=0`, `-trimpath`, `-ldflags="-s -w"`) and ships it on `alpine:3.20` with only CA certificates, running as unprivileged uid 10001 on port 9090.

Observability. Logs are structured JSON (`log/slog`). Requests are logged with method, path and status, never the query string. Refusals are logged with the precise reason that the invitee's page withholds on purpose. The nonce never reaches the logs, except through the opt-in log mail provider. `/health` reports liveness and the mail provider but does not check Keycloak, so it is a liveness probe, not a readiness probe.

12.2 Cost per operation

Because Keycloak is the system of record, each operation's cost is mostly Keycloak round trips. Counting them from the code [Code]:

Operation	Keycloak calls	Mail	Notes
Issue, new address	5	1	introspect, search, create, write attributes, read back; +2 with `DEFAULT_ROLE`
Re-issue (expired or `force`)	5	1	introspect, search, fetch, write, read back
Accept, success	4	–	search, fetch, enable-and-burn, read back
Accept, unknown address	1	–	search only (finding F2)
Accept, existing address, wrong nonce	2	–	search, fetch; no write (T6)
List	2	–	introspect, attribute query (capped at 1,000)
Revoke	4	–	introspect, search, fetch, delete

An admin token fetch is added at most once per token lifetime, minus the 30-second safety margin. Each of these is a single-digit number of calls per human action. Invitations happen at human pace, so latency and throughput are bounded by Keycloak, not by the service.

12.3 Scaling

Apart from per-process token caches the service is stateless, so replicas scale horizontally with no coordination. The limits are elsewhere: listing is capped at 1,000 results (known gap 4), and finding F5's races grow slightly more likely with more replicas.

12.4 Behaviour when dependencies fail

Failure	Behaviour
Configuration invalid	Refuses to boot; lists every problem
Keycloak unreachable at boot	Refuses to boot (60-second provisioning budget)
IdP metadata missing	Boots, with a warning that nobody can complete SSO
IdP metadata without a signing certificate	Refuses to boot
Keycloak unreachable at runtime (admin)	503 when introspection fails; 500 otherwise. Never falls open
Keycloak unreachable at runtime (accept)	400 “cannot be used” page (finding F6)
Attribute write fails during create	Half-built user deleted; 500
Mail provider fails	500; invitation stored; retry with `force`
Default role cannot be assigned	Logged; invitation still sent
`SIGTERM`	Stops accepting; drains for up to 15 seconds

13 Value Proposition

There are no deployment metrics in the repository, so this section argues value from design properties the code and tests support, not from usage numbers.

13.1 Who it is for

The service fits wherever a realm is federated to a large directory but an application is meant for a deliberate subset of it: internal tools that 30 of 5,000 employees should see; staged rollouts and pilots that admit people one invitation at a time; partner and contractor directories where only named individuals are admitted; and explicit-grant environments where access must follow a named person’s decision. For a formal compliance regime, the missing audit trail (known gap 3) needs closing first.

13.2 What it saves

No code inside the identity server. A custom authenticator is Java compiled against Keycloak’s internal APIs. It needs a build pipeline, Java expertise, a compatibility check at every upgrade and a redeploy of the identity server itself. Here the gate is *configuration* applied and repaired through the public Admin API. The risk an upgrade carries is narrowed to one documented behaviour (REQUIRED discards ALTERNATIVE), and the configuration side of that is pinned by tests.

No second system of record. There is no database to provision, back up, migrate, secure or keep consistent with the directory. Invitations cannot drift out of sync with users, because they *are* users.

A small trust surface. The service holds realm-admin credentials, so its code is part of the identity system’s trusted computing base. That is 2,560 lines of standard-library Go, and the rules that decide security outcomes fit in a 166-line package with no I/O, small enough to audit in one sitting.

Safe defaults that cannot be skipped. Signature validation cannot be switched off, misconfiguration stops the boot, admin authorisation fails closed, flow requirement drift is corrected on restart, and re-invitation never silently breaks a live link or resets an active account. Each removes a way for a busy operator to make a mistake nobody notices.

13.3 Cost comparison

Table 2 compared the options on capability. Table 6 compares them on what each one costs to own.

Approach	Code in Keycloak	Extra state	Upgrade coupling	Licence
Manual pre-creation	none	none	configuration only	–
Custom Java SPI	yes	your choice	internal Java APIs	yours
`keycloak-orgs`	extension	extension tables	extension release cadence	ELv2
Keycloak Organizations	none (first-party)	Keycloak	first-party	Apache-2.0
This project	**none**	**none** (user attributes)	**documented flow semantics; Admin API**	**Apache-2.0**

Table 6. What each approach costs to own.

13.4 When not to use it

- **Organizations fits your tenancy.** It is first-party and maintained by the Keycloak team. Prefer it.
- **Your IdP lets users choose their asserted email** (assumption A1). The identity half of the gate is then weak.
- **You need an audit trail, or lists in the tens of thousands, today.** Both need the store that the `Directory` seam anticipates but that does not exist yet.
- **You need several realms per instance.** The service is one instance per realm by design.
- **You need to control access after admission.** The gate controls first entry. Ongoing authorisation remains Keycloak roles and the application's own checks.

13.5 Engineering practices on display

For readers judging the engineering, each of these can be checked in the code: a functional core with clock-injected, table-tested rules; read-after-write verification against a system whose writes can fail silently; fail-closed defaults applied consistently; ordering arguments made explicit (one write for enable-and-burn, store before send, provision before listen); a threat model with non-goals and known gaps; tests that pin an external system's configuration, with a firm rule against mocking Keycloak; and comments that explain *why*.

14 Roadmap and Conclusion

14.1 Prioritised roadmap

These items address every finding and known gap, ordered by value per unit of effort. Sizes are rough judgements (S: hours, M: a day or two, L: a week or more).

#	Item	Closes	Size
1	Behavioural SAML end-to-end test against the bundled mock IdP	§11 gap; strengthens T1	M
2	Confirmation page: `GET` renders, `POST` redeems; nonce in the URL fragment	F3, T10, gap 2	S
3	Check the nonce before expiry; drop the redundant fetch; extend the indistinguishability test	F1, F2	S

#	Item	Closes	Size
4	Map infrastructure failures on accept to a 503 page	F6	S
5	Reconcile flow structure; check every IdP's flow binding at boot	F7, A5	M
6	Service account with `manage-users` and `manage-identity-providers`	gap 5, A4	S
7	Audience or client check on introspected tokens	F4	S
8	Complete the demo: provision the introspection client and role in dev, wire the mock IdP's metadata, add a stub application	§11 demo gaps	S
9	Restrict revoke to pending invitations; unit tests for revoke, `User.Record` and the domain allow-list	F8; coverage gaps	S
10	Rate limiting on admin and accept endpoints	gap 1; reduces F2 sampling	S
11	Invitation store behind `Directory` for audit and large-scale listing	gaps 3, 4; F5	L
12	Reaper for expired and activated-but-never-linked accounts	gap 6	M

Do items 1–4 first. Item 1 turns the central claim from "configured correctly and documented by Keycloak" into "observed to hold"; items 2–4 are a few lines each.

14.2 Conclusion

Federation decides who someone is. It should not, by default, decide whether they are admitted. `keycloak-invite-gate` separates the two with what Keycloak already offers: a disabled user that stands for an administrator's intent, and a first-broker-login flow that links to an existing account but never creates one. Reaching the application takes two independent facts, an invitation and a verified assertion from the identity provider, with no Java in the identity server and no database beside it. The core mechanism turns out to be a configuration Keycloak itself documents, which is the best available basis for durability.

The review behind this paper found no way for an uninvited identity to sign in. It did find gaps between some claims and the code: the end-to-end suite verifies configuration rather than behaviour, one failure page can be told apart from the others, and the redemption link is exposed to mail scanners. Each has a small, well-understood fix, and the roadmap orders them. With those closed, the project would be more than a clear demonstration of the mechanism: it would be a component an operator could adopt with confidence.

A Configuration Reference

Variable	Default	Purpose
`KEYCLOAK_URL`	`http://localhost:8080`	Keycloak as reached by the service
`KEYCLOAK_EXTERNAL_URL`	= `KEYCLOAK_URL`	Keycloak as reached by browsers and the IdP
`KEYCLOAK_REALM`	`invite-gate`	The realm the service manages (must not be `master`; not enforced)
`KEYCLOAK_ADMIN_USER` / `_PASSWORD`	*required*	Master-realm admin for the Admin API
`INTROSPECTION_CLIENT_ID` / `_SECRET`	*required*	Confidential client for token introspection
`ADMIN_ROLE`	`invite-admin`	Realm role required on admin endpoints
`IDP_ALIAS`	`corp`	IdP alias; also the `kc_idp_hint`
`IDP_METADATA_FILE`	(none)	IdP SAML metadata; without it SSO is not configured
`SAML_SP_ENTITY_ID`	`keycloak-<realm>`	Entity id registered with the IdP

Variable	Default	Purpose
`PUBLIC_URL`	*required*	This service's external root, used for links
`APP_LOGIN_URL`	*required*	Where an accepted invitation redirects
`INVITE_TTL`	`15m`	Invitation validity (Go duration syntax)
`ALLOWED_DOMAIN`	(any)	Restrict invitations to one email domain
`DEFAULT_ROLE`	(none)	Realm role granted at invitation time
`MAIL_PROVIDER`	`smtp`	`smtp`, `zoho` or `log`
`MAIL_FROM`	*required*†	Sender address
`SMTP_HOST` / `_PORT`	`localhost` / `1025`	SMTP relay
`SMTP_USERNAME` / `_PASSWORD`	(none)	Enables PLAIN auth when set
`SMTP_STARTTLS`	`false`	Upgrade the connection after EHLO
`ZOHO_CLIENT_ID`, `_SECRET`, `_REFRESH_TOKEN`, `_ACCOUNT_ID`	*required*‡	Zoho OAuth and account
`ZOHO_ACCOUNTS_URL` / `_API_URL`	*required*‡	Region hosts; no default on purpose
`ALLOW_LOG_MAIL_PROVIDER`	(none)	Must be `yes` to select `log`
`PORT` / `LOG_LEVEL`	`9090` / `info`	Listener and log verbosity

†For `smtp` and `zoho`. ‡For `zoho`.

B Glossary

Auto-link `idp-auto-link`: binds a brokered identity to an existing user without prompting.

Burn Deleting the nonce hash and expiry so that the link can never be redeemed again.

Introspection RFC 7662: asking the issuer whether a token is active, and what it carries.

`kc_idp_hint` A request parameter that sends the user straight to a named IdP.

Shadow user The disabled Keycloak user that represents a pending invitation.

References

[1] Keycloak. *Server Administration Guide*, sections "Execution requirements", "Detect existing user first login flow" and "Understanding managed and unmanaged attributes". https://www.keycloak.org/docs/latest/server_admin/index.html

[2] Keycloak source, tag 26.5.0 (authentication flow, broker authenticators, admin flow management, token introspection). https://github.com/keycloak/keycloak/tree/26.5.0

[3] Keycloak. *Upgrading Guide*, 24.0.0 (user profile always enabled) and 26.0.0 (bootstrap admin variables). https://www.keycloak.org/docs/latest/upgrading/index.html

[4] Keycloak. *Release Notes*, 26.0.0 (Organizations fully supported) and 26.5.0 (persistent organization invitations). https://www.keycloak.org/docs/latest/release_notes/index.html

[5] Keycloak. *Securing Applications: OIDC layers*, token introspection endpoint. https://www.keycloak.org/securing-apps/oidc-layers

[6] J. Richer, Ed. *OAuth 2.0 Token Introspection*. RFC 7662, October 2015.

[7] M. Nottingham, E. Wilde, S. Dalal. *Problem Details for HTTP APIs*. RFC 9457, July 2023 (obsoletes RFC 7807).

[8] Phase Two. `keycloak-orgs`: single realm, multi-tenancy for SaaS apps. https://github.com/p2-inc/keycloak-orgs

[9] `moeakwak/keycloak-registration-with-invite-code`: registration form action requiring an administrator-issued invite code. https://github.com/moeakwak/keycloak-registration-with-invite-code

[10] Microsoft. *Safe Links in Microsoft Defender for Office 365*. https://learn.microsoft.com/en-us/defender-office-365/safe-links-about

[11] Stytch. *Protected email magic links*. https://stytch.com/docs/b2b/guides/magic-links/protected-eml

[12] Supabase. *OTP verification failures: token has expired*. https://supabase.com/docs/guides/troubleshooting/otp-verification-failures-token-has-expired-or-otp_expired-errors-5ee4d0